A

Project Report on

ML-BASED COLLEGE PREDICTOR TELEGRAM BOT INDIAN ENGINEERING COLLEGES

Submitted for partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

by

P. PRABHU **22K81A72A2**

Under the Guidance of

Mr. S. Yellappa

ASSISTANT PROFESSOR

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

St. MARTIN'S ENGINEERING COLLEGE

UGC Autonomous
Affiliated to JNTUH, Approved by AICTE
Accredited by NBA & NAAC A+, ISO 21001:2018 Certified
Dhulapally, Secunderabad-500 100
www.smec.ac.in

MAY – 2026

Certificate

This is to certify that the project entitled **"ML-Based College Predictor Telegram Bot Indian Engineering Colleges"** is being submitted by P. PRABHU (22K81A72A0), in the partial fulfilment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** is a Bonafide work carried out by them. The results embodied in this report have been verified and found satisfactory.

Signature of Guide

Mr. S. YELLAPPA

Assistant professor

Department of AI&DS

Signature of HOD

Dr. N. KRISHNAIAH

Associate Professor & HOD

Department of AI&DS

Internal Examiner

External Examiner

Date:

Place:

St. Martin's ENGINEERING COLLEGE

UGC Autonomous

NBA & NAAC A+ Accredited

Dhulapally, Secunderabad - 500 100

www.smec.ac.in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE (AI&DS)

DECLARATION

We the students of '**Bachelor of Technology** from the **Department of Artificial Intelligence and Data Science'**, session: 2022 - 2026**, St. Martin's Engineering College, Dhulapally, Kompally, Secunderabad,** hereby declare that the work presented in this Project Work entitled "**ML-based college predictor telegram bot Indian engineering colleges"** is the outcome of our bonafide work and is correct to the best of our knowledge and this work has been undertaken taking care of Engineering Ethics. The result embodied in this project report has not been submitted to any university for the award of any degree.

P. PRABHU 22K81A72A0

ACKNOWLEDGEMENT

The satisfaction and euphoria accompanying the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

First and foremost, we would like to express our deep sense of gratitude and indebtedness to our College Management for their kind support and permission to use the institute's facilities.

We especially would like to express our deep gratitude and indebtedness to **Prof. Dr. KASA RAVINDRA**, Director and Principal, St. Martin's Engineering College, Dhulapally, for permitting us to undertake this project.

We are also thankful to **Dr. N. KRISHNAIAH**, Head of the Department, and Project Coordinator Mr.S.Yellappa, Assistant Professor, Department of Artificial Intelligence & Data Science, St. Martin's Engineering College, Dhulapally, Secunderabad, for their support and guidance throughout our project.

We would like to express our sincere gratitude and indebtedness to our project supervisor, **Mr. S. Yellappa**, Associate Professor, Artificial Intelligence & Data Science, St. Martin's Engineering College, Dhulapally, for his support and guidance throughout our project.

Finally, we express thanks to all those who have helped us complete this project. Furthermore, we would like to thank our family and friends for their moral support and encouragement. We express our thanks to all those who have helped us in completing the project.

J. SUMANTH 22K81A7280

ABSTRACT

The **ML-Based College Predictor Telegram Bot for Indian Engineering Colleges** is designed to assist students in selecting suitable colleges based on their entrance exam ranks. In India, the counselling process can be confusing due to the large number of colleges, varying cutoff trends, and lack of clear guidance. This project provides a simple and effective solution by offering instant college predictions through an automated chatbot.

The system is developed using Python and Flask for backend processing. It is integrated with Telegram using the python-telegram-bot, allowing users to interact easily through a familiar messaging platform. The chatbot collects user inputs such as rank, category, gender, and preferred branch

The core functionality of the system is based on historical cutoff data of Indian engineering colleges. The system analyzes this data and compares it with the user's input to predict colleges where the student has a higher chance of admission. It mainly uses filtering techniques and basic machine learning concepts to generate accurate and relevant results quickly.

The project offers advantages such as ease of use, quick response time, and accessibility without requiring any special software. It reduces the effort needed for manual research and helps students make better decisions. In the future, the system can be enhanced by adding real-time data updates, advanced machine learning models, and additional features like location-based filtering and college ratings.

UGC AUTONOMOUS

LIST OF FIGURES

Figure No.	Figure Title	Page No.
3.2	Architectural Block Diagram of Proposed System	05
4.3.1	System Architecture	10
4.3.2	Data Flow Diagram	11
4.3.4	Use Case Diagram	12
4.3.5	Class Diagram	13
4.3.6	Sequence Diagram	14
4.3.7	Activity Diagram	14

LIST OF TABLES

Table No.	Table Title	Page No.
3.3	System Configuration	56
4.6.2	Unit Testing	18
4.6.3	Integration Testing	19
4.6.4	System Testing	20
4.6.5	Performance Testing	20
4.6.6	Model performance Testing	21
6.1	Dataset Description	45
6.2	GAN Training Results	45

6.3	Confusion Matrix Analysis	46
6.5	Realtime Interference Performance	47

LIST OF ACRONYMS AND DEFINITIONS

S.NO	ACRONYM	DEFINITION
01	LLM	Large Language Model
02	CI/CD	Continuous Integration/Continuous Deployment
03	CLI	Command Line Interface
04	API	Application Programming Interface
05	GAN	Generative Adversarial Networks

CONTENTS

ACKNOWLEDGEMENT i

ABSTRACT ii

LIST OF FIGURES iii

LIST OF TABLES iv

LIST OF ACRONYMS AND DEFINITIONS iv

CHAPTER 1 INTRODUCTION 1

1.1 Objective 1

1.2 Overview 2

CHAPTER 2 LITERATURE SURVEY 3

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 4

3.1 Existing System 4

3.2 Proposed System 5

3.3 System Configuration 7

CHAPTER 4 SYSTEM REQUIREMENTS AND SPECIFICATIONS 8

4.1 Database 8

4.2 GAN Model 9

4.3 Design 9

4.3.1 System Architecture 10

4.3.2 Data Flow Diagram 11

4.3.3 UML Diagram 11

4.3.4 Use Case Diagram 11

4.3.5 Class Diagram 12

4.3.6 Sequence Diagram 13

4.3.7 Activity Diagram 14

4.4. Modules 15

4.4.1 Modules Description 15

4.5 System Requirements 15

4.5.1 Hardware Requirements 16

4.5.2 Software Requirements 16

4.6 Testing 17
4.6.1 Testing Strategy 17
4.6.2 Unit Testing 17
4.6.3 Integration Testing 18
4.6.4 System Testing 19
4.6.5 Performance Testing 20
4.6.6 Model Performance Testing 21

CHAPTER 5 SOURCE CODE 22
CHAPTER 6 EXPERIMENTAL RESULTS 45
CHAPTER 7 CONCLUSION AND FUTURE ENHANCEMENT 48
7.1 Conclusion 48
7.2 Future Enhancement 49

REFERENCES 50

CHAPTER 1

INTRODUCTION

1.1 Objective

The primary objective of the ML-Based College Predictor Telegram Bot for Indian Engineering Colleges is to develop an intelligent, automated, and user-friendly system that assists students in predicting suitable engineering colleges based on their entrance examination rank and related parameters. This project addresses the common challenges students face during the counselling process by providing clear, data-driven recommendations through an easy-to-use chatbot interface, reducing confusion and improving decision-making.

Another key objective is to effectively utilize historical admission data to generate reliable predictions. The system analyzes past cutoff ranks influenced by factors such as category, gender, branch, and institution. By applying data filtering techniques and basic machine learning concepts, it identifies patterns and trends to estimate admission chances. This ensures that the results are more accurate and dependable compared to traditional manual methods, while also reducing the time and effort required for research.

The project also focuses on accessibility, real-time interaction, and future scalability. By integrating the system with Telegram and developing the backend using Python and Flask, users can easily interact with the bot and receive instant responses. Additionally, the system is designed to be efficient, flexible, and extendable, allowing future improvements such as advanced machine learning models, real-time data updates, and additional features like location-based filtering and college ranking.

1.2 Overview

The ML-Based College Predictor Telegram Bot for Indian Engineering Colleges is an intelligent system designed to help students choose suitable colleges based on their entrance exam performance. In India, the college selection process is often complex due to the large number of institutions, varying cutoff trends, and lack of clear guidance. This project provides a simple and effective solution by offering instant predictions through an interactive chatbot, helping students make informed decisions about their future.

The system is built using Python for backend development and Flask to handle server-side operations. It is integrated with Telegram using the python-telegram-bot, allowing users to interact with the system in a conversational manner. The bot collects user inputs such as rank, category, gender, and preferred branch in a step-by-step format.

The core functionality of the project relies on analyzing historical cutoff data of Indian engineering colleges. The system processes this data using filtering techniques and basic machine learning concepts to predict colleges where the student has a higher probability of admission. By comparing the user's rank with previous years' closing ranks, the system generates a list of eligible colleges, making the prediction process fast and reliable.

Overall, the project is designed to be simple, accessible, and efficient. It reduces the need for manual research and provides quick results through a widely used messaging platform. The system can be further enhanced by incorporating real-time data updates, advanced machine learning algorithms, and additional features such as location-based filtering and college rankings, making it a scalable and practical solution for students during the admission process.

UGC AUTONOMOUS

CHAPTER 2
LITERATURE SURVEY

Several existing systems and research studies have focused on predicting suitable colleges for students based on entrance exam ranks and academic performance. Traditional methods mainly rely on manual analysis of previous years' cutoff data, counselling brochures, and guidance from coaching institutes. These approaches are often time-consuming, less accurate, and prone to human error. Recent studies have introduced automated systems that use data-driven techniques to simplify the college selection process and improve prediction accuracy.

With the advancement of machine learning, tools such as scikit-learn have been widely used to develop predictive models in the education domain. Algorithms like Decision Trees, Random Forest, and K-Nearest Neighbors have been applied to analyze historical admission datasets and identify patterns in cutoff trends. These models help in classifying colleges based on student eligibility and predicting admission chances, offering a more efficient alternative to manual methods.

In addition to prediction systems, chatbot-based applications have gained popularity for providing interactive and user-friendly services. Platforms like Telegram combined with frameworks such as python-telegram-bot enable real-time communication between users and systems. Research shows that chatbot interfaces improve accessibility and user engagement, especially for students who prefer simple and quick interactions over complex web applications.

However, many existing systems lack real-time updates, scalability, and personalization features. Some applications provide only static information without adapting to user-specific inputs such as category, gender, or preferences. This project addresses these gaps by integrating machine learning techniques with a chatbot interface to deliver personalized, fast, and reliable college predictions. It also emphasizes ease of use, efficient data processing, and the potential for future enhancements, making it a more practical solution compared to earlier approaches.

UGC AUTONOMOUS

CHAPTER 3
SYSTEM ANALYSIS

3.1 Existing System

The existing system for college prediction mainly relies on manual analysis of cutoff data from various sources, making the process time-consuming and difficult for students. It lacks accuracy and personalization, as students often depend on guesswork or incomplete information without proper consideration of factors like category, gender, and branch preference. Additionally, there is no real-time interaction or automated support, forcing students to spend significant time researching and comparing data from multiple platforms. The absence of a centralized and user-friendly system further complicates the decision-making process, leading to confusion and unreliable results.

However, these practices have a few drawbacks

Disadvantages of the Existing System

- **Manual and Time-Consuming Process** – Students must check multiple sources like PDFs and websites.
- **Lack of Accuracy** – Predictions are based on guesswork or incomplete data.
- **No Personalization** – Does not consider factors like category, gender, or branch preference.
- **No Real-Time Interaction** – No instant response or automated guidance system.
- **Scattered and Complex Information** – Data is difficult to understand and not available in one place..

3.2 Proposed System

The proposed system is an **ML-Based College Predictor Telegram Bot** designed to provide a fast, accurate, and user-friendly solution for students selecting engineering colleges. The system uses historical cutoff data and basic machine learning techniques to predict suitable colleges based on inputs such as rank, category, gender, and preferred branch. It is developed using **Python** and **Flask**, and integrated with **Telegram** through the **python-telegram-bot** to enable real-time interaction. Unlike traditional methods, this system provides instant, personalized results in a simple conversational format, reducing manual effort and improving decision-making. It also ensures better accessibility, centralized information, and can be further enhanced with advanced machine learning models, real-time data updates, and additional filtering options.

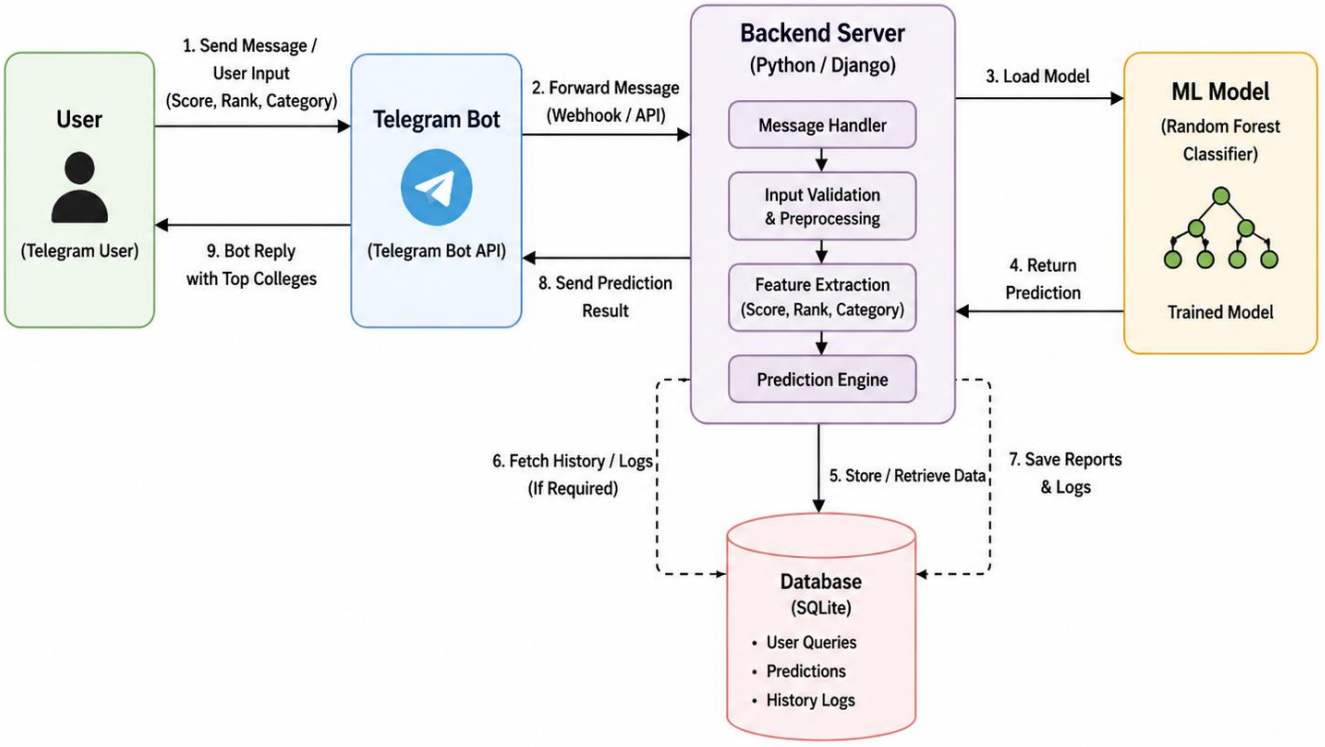


Figure 3.2. Architectural Diagram of Proposed System.

Advantages of the Proposed System:

1. **Fast and Real-Time Results**
   The system processes user inputs instantly and provides immediate college predictions. This real-time response helps students quickly make decisions without waiting or manually searching through large datasets.

2. **Improved Accuracy**
   By using historical cutoff data along with filtering and basic machine learning techniques, the system provides more reliable and data-driven predictions compared to guesswork or manual analysis.

3. **Personalized Output**
   The system considers important user-specific factors such as rank, category, gender, and preferred branch. This ensures that the results are tailored to each individual student's profile.

4. **Easy-to-Use Interface**
   The chatbot is integrated with Telegram, allowing users to interact in a simple conversational way. This makes the system accessible even for users with no technical knowledge.

5. **Reduces Effort and Dependency**
   The system eliminates the need for manual research and reduces reliance on coaching centers or external sources. Students can independently access accurate information and make informed decisions.

UGC AUTONOMOUS

3.3 System Configuration

The system configuration for the **ML-Based College Predictor Telegram Bot** includes both hardware and software components required to develop and run the application efficiently. The project is developed using Python with Flask as the backend framework, and it integrates with Telegram using **the python-telegram-bot**. The system requires a basic computer with internet connectivity for development and deployment. It uses a dataset **(CSV or database)** containing historical cutoff data, and can be deployed on cloud platforms for continuous access. The configuration is lightweight, making it easy to run on standard systems without high-end hardware requirements..

Library	Version	Purpose
Django	4.2	Web framework and ORM
djangorestframework	3.14	RESTful API development
TensorFlow	2.13	GAN and DNN model development
Keras	2.13	High-level neural network API
scikit-learn	1.3	Preprocessing, metrics, baseline models
pandas	2.0	Data manipulation and analysis
NumPy	1.25	Numerical computing
matplotlib / seaborn	3.7 / 0.12	Data visualization
imbalanced-learn	0.11	SMOTE and resampling utilities
joblib	1.3	Model serialization
psycopg2	2.9	PostgreSQL adapter for Django
Chart.js	4.3	Frontend data visualization

Table 3.3 System Configuration

UGC AUTONOMOUS

CHAPTER 4
SYSTEM REQUIREMENTS AND SPECIFICATIONS

4.1 Dataset

The dataset used in the ML-Based College Predictor Telegram Bot plays a crucial role in determining the accuracy and reliability of the predictions. It consists of historical admission data collected from previous years' counselling records of Indian engineering colleges. This data includes important attributes such as college name, branch, category, gender, and closing rank. These parameters are essential because admission cutoffs vary significantly based on these factors. By organizing this information into a structured format such as a CSV file or database, the system can efficiently process and analyze the data.

Before using the dataset, it undergoes **data preprocessing**, which involves cleaning and organizing the raw data. This includes handling missing values, ensuring consistency in category names, and formatting ranks properly. Preprocessing improves the quality of the dataset and ensures that the prediction logic works accurately. The dataset is then used by the backend system, developed using Python, where it is loaded and processed using libraries such as pandas. Properly structured data allows faster filtering and comparison operations during prediction.

The working principle of the dataset is based on comparing the user's input rank with the historical **closing ranks** of colleges. If the user's rank is less than or equal to the closing rank of a particular college and branch, the system considers it as a possible option. This method follows a data-driven approach rather than relying on assumptions. In some cases, basic machine learning techniques can also be applied to identify patterns and improve prediction accuracy, but simple filtering logic itself provides effective results for this type of problem.

Overall, the dataset acts as the foundation of the entire system. Its quality, size, and accuracy directly impact the performance of the college predictor bot. A well-maintained dataset ensures reliable outputs, while continuous updates with new counselling data can further enhance the system's effectiveness. This makes dataset management an important aspect of the project, enabling the system to deliver meaningful and trustworthy recommendations to students.

4.2 GAN Model

The Generative Adversarial Network (GAN) is an advanced machine learning model that can be optionally applied in the ML-Based College Predictor Telegram Bot to enhance data quality and prediction capability. A GAN consists of two main components: a Generator and a Discriminator. The Generator creates synthetic data similar to real historical cutoff data, while the Discriminator evaluates whether the data is real or generated. Through this adversarial process, the model improves over time and produces realistic data patterns.

In the context of this project, GAN can be used to **generate additional synthetic cutoff data** when real historical data is limited or incomplete. For example, if certain categories or branches have fewer records, the GAN model can create realistic data samples based on existing patterns. This helps in improving the dataset size and diversity, which can lead to better prediction performance. It is especially useful when working with sparse or unbalanced datasets.

The GAN model is typically implemented using deep learning frameworks like TensorFlow or PyTorch. However, it is important to note that GANs are complex and computationally intensive compared to basic machine learning models. For a student-level project, GAN is considered an **advanced enhancement** rather than a core requirement. The primary system can work effectively using simple filtering or basic ML techniques without GAN.

Overall, the use of GAN in this project is optional and mainly aimed at improving data generation and model robustness. While it adds innovation and research value to the project, it also increases complexity. Therefore, it should be included only if there is sufficient understanding and time to implement it properly; otherwise, a simpler approach is more practical and reliable for this application.

4.3 Design

T The **ML-Based College Predictor Telegram Bot** follows a simple and modular design that connects the user interface, backend processing, and dataset to deliver accurate college predictions. The system is designed in such a way that each component performs a specific function, making it easy to develop, test, and maintain. The overall design ensures smooth communication between the user and the prediction system through an automated chatbot.

4.3.1 System Architecture

The system architecture of the ML-Based College Predictor Telegram Bot is based on a client-server model that enables efficient interaction between the user and the system. The user communicates with the application through the Telegram chatbot, which acts as the front-end interface. It collects inputs such as rank, category, gender, and preferred branch in a simple conversational manner. The chatbot is integrated using the python-telegram-bot, which facilitates smooth communication between the user and the backend server.

The backend is developed using Python and the Flask framework, where all processing and prediction tasks are performed. It receives user inputs, processes them, and applies filtering or basic machine learning techniques using historical cutoff data stored in a dataset. The system then generates a list of suitable colleges and sends the response back to the user through the chatbot. This modular architecture ensures fast performance, easy maintenance, and scalability for future enhancements.

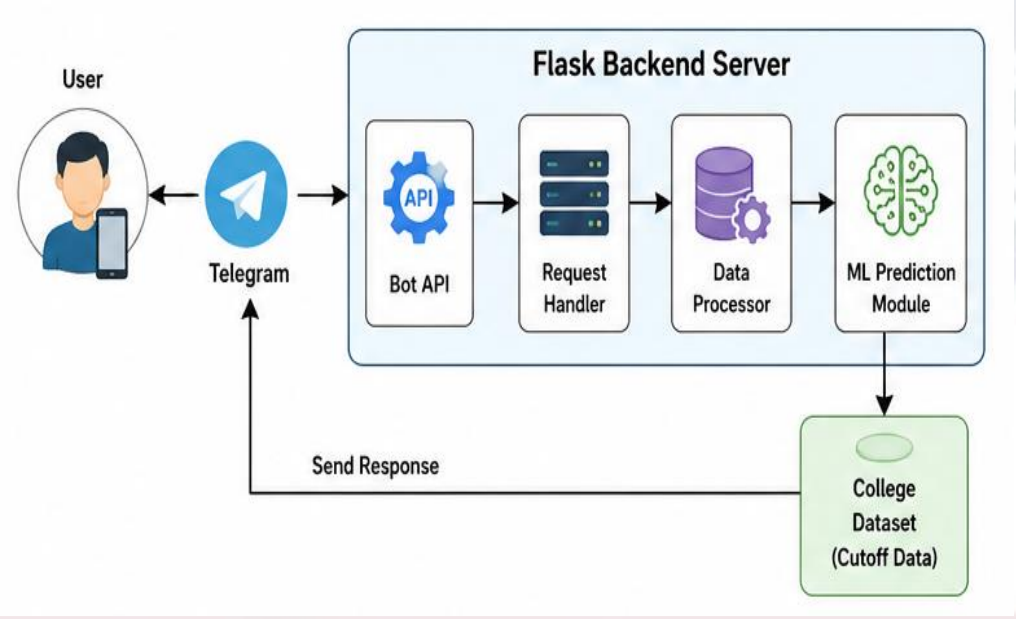


Fig: 4.3.1 System Architecture

4.3.2 Data Flow Diagram

The Data Flow Diagram (DFD) of the **ML-Based College Predictor Telegram Bot** shows how user data is processed through the system. The user provides inputs like rank and category via Telegram, which are received by the bot and sent to the backend developed using Python and Flask. The backend processes the input and compares it with the dataset containing historical cutoff data. Based on this, the prediction module generates a list of suitable colleges, and the results are sent back to the user through the chatbot. This ensures a simple, fast, and efficient data flow within the system.

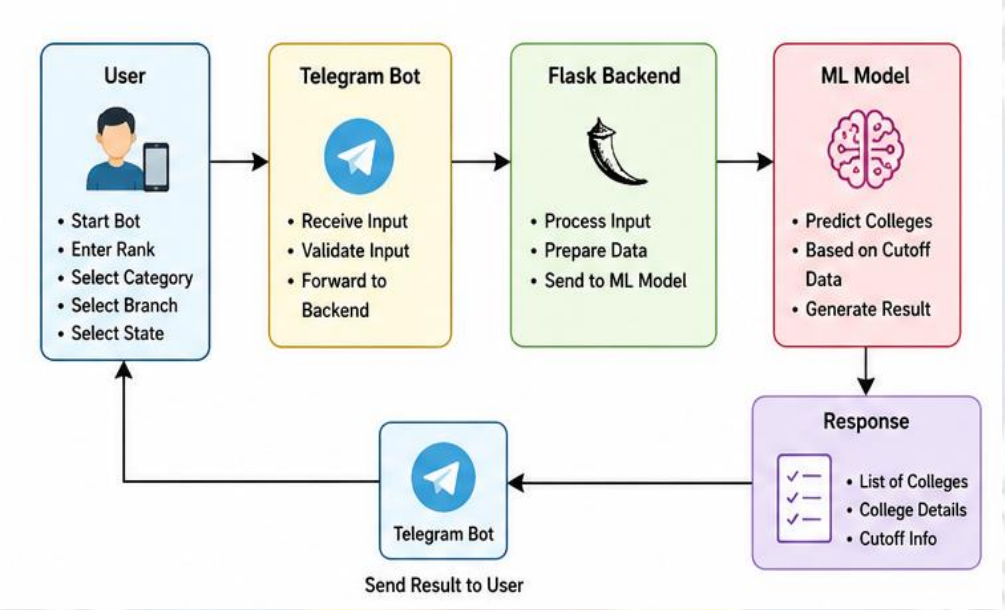


Fig: 4.3.2 Data Flow Diagram

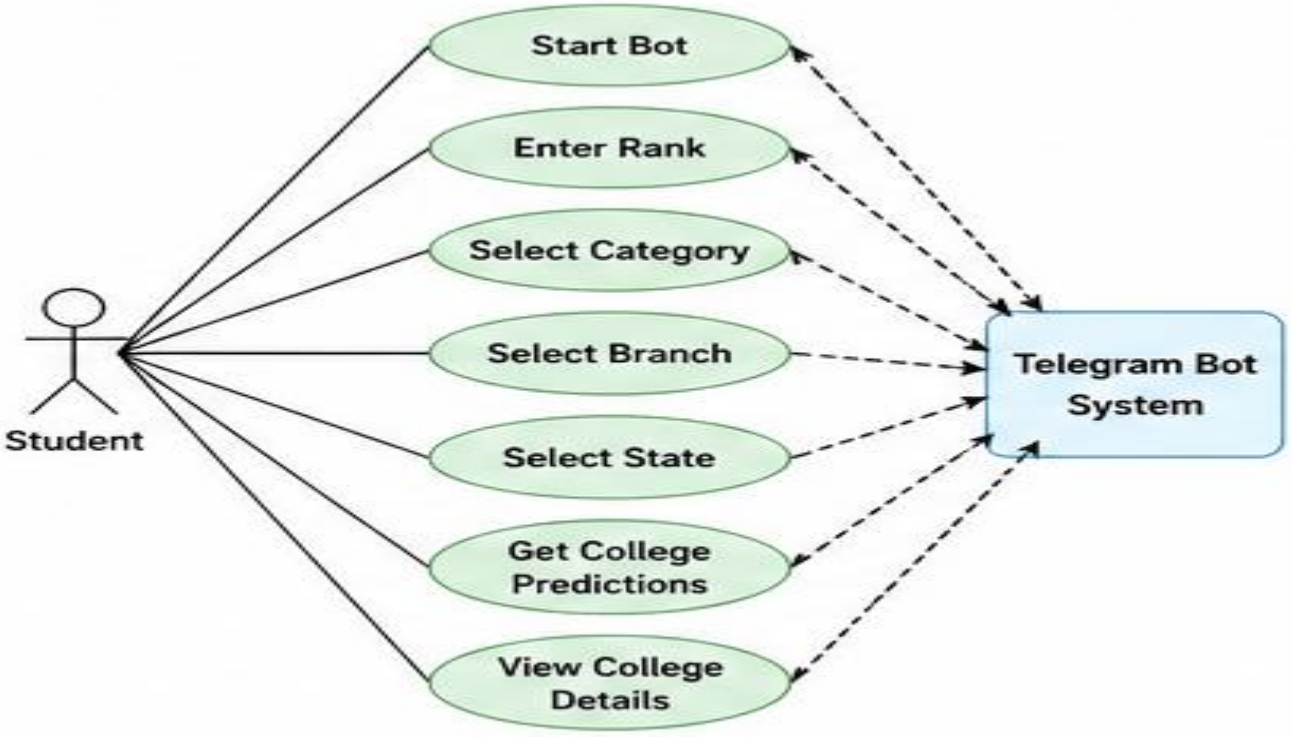
4.3.3 UML Diagram

UML (Unified Modeling Language) diagrams are used to visually represent the design and structure of the **ML-Based College Predictor Telegram Bot**. They help in understanding how different components of the system interact with each other. Common UML diagrams used in this project include Use Case diagrams (to show user interactions), Sequence diagrams (to show the flow of communication), and Activity diagrams (to represent the workflow of the system). These diagrams simplify complex processes, improve system clarity, and help in better planning, development, and documentation of the project.

4.3.4 Use Case Diagram

A Use Case Diagram is a type of UML diagram that represents the interaction between users (actors) and the system. In the **ML-Based College Predictor Telegram Bot**, it helps to visualize how a student interacts with the chatbot to get college predictions. The diagram focuses on what the system does rather than how it does it, showing the different functionalities (use cases) such as entering details, processing data, and displaying results. It improves understanding of system requirements and user interactions in a simple graphical form.

Fig: 4.3.4 Use Case Diagram



4.3.5 Class Diagram

A Class Diagram is a UML diagram that represents the static structure of the ML-Based College Predictor Telegram Bot by showing the system's classes, their attributes, methods, and relationships. It acts as a blueprint for the system, helping developers understand how different components are organized and how they interact with each other. The main classes in this project include User, BotHandler, BackendServer, PredictionModel, and Dataset, each responsible for a specific function in the system.

The **User** class stores input details such as rank, category, and preferences, while the **BotHandler** class manages communication through python-telegram-bot. The **BackendServer**, built using Flask, processes requests and connects different modules. The **PredictionModel** handles the logic for predicting colleges based on the dataset, and the **Dataset** class manages historical cutoff data. These classes are interconnected, ensuring smooth data flow and efficient system operation, making the design modular, scalable, and easy to maintain.

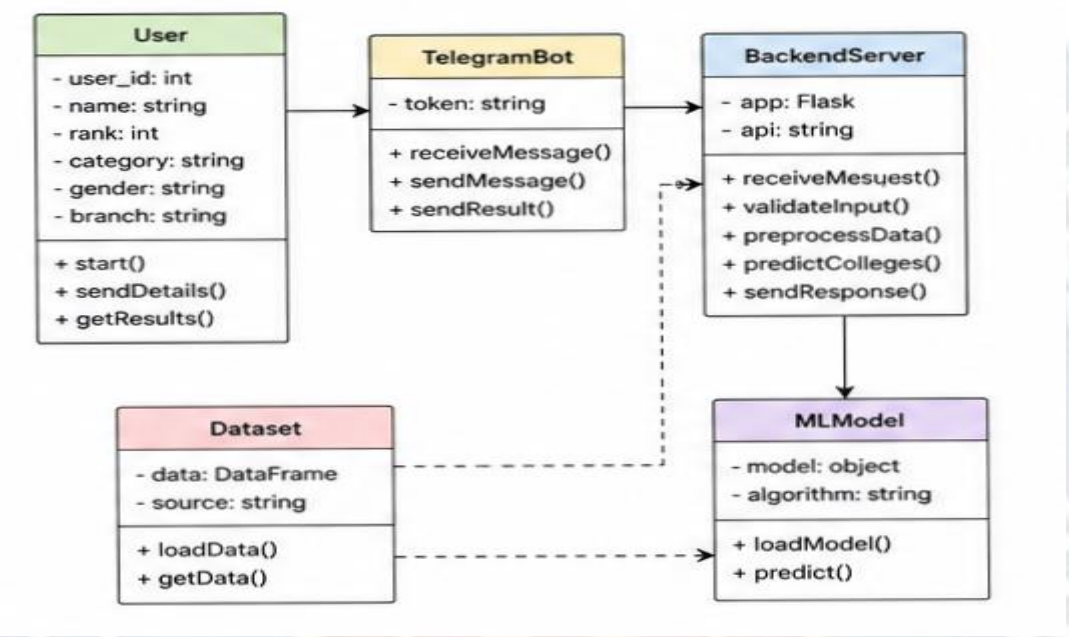
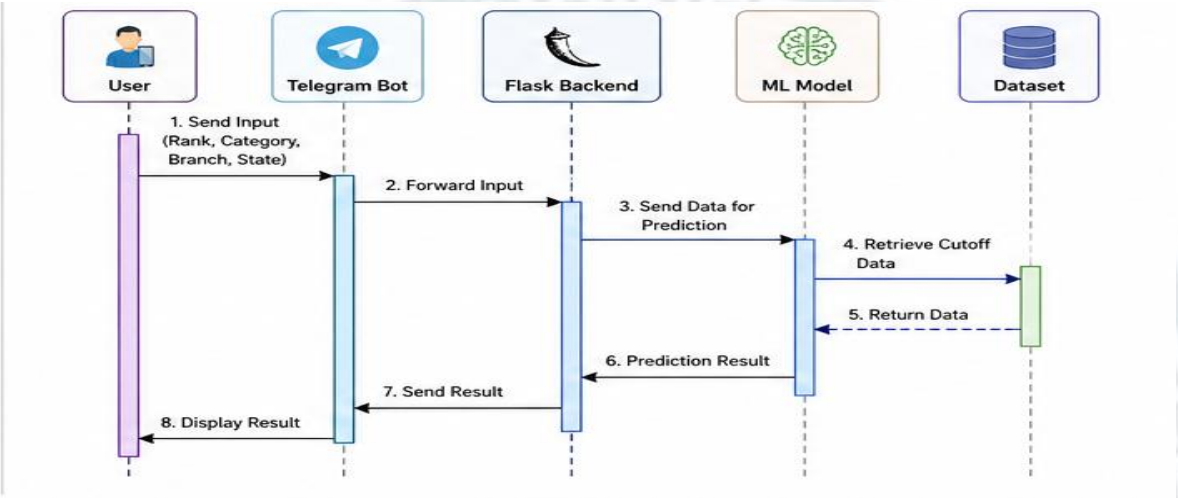


Fig: 4.3.5 Class Diagram

4.3.6 Sequence Diagram

A Sequence Diagram is a type of UML diagram that shows how different components of a system interact with each other in a specific sequence over time. In the ML-Based College Predictor Telegram Bot, it illustrates the step-by-step flow of messages between the user, chatbot, backend server, and prediction module. It helps in understanding how requests are processed and how responses are generated in a time-ordered manner.

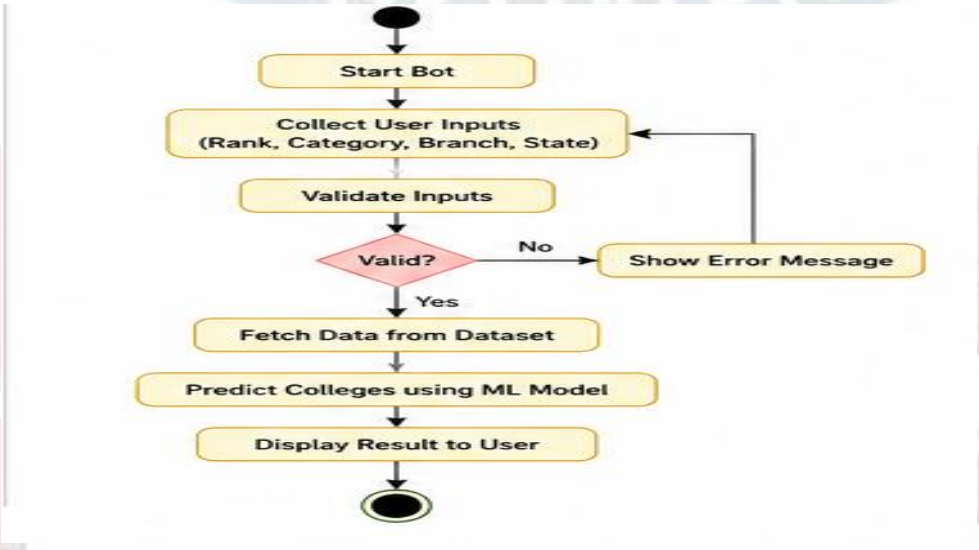
In this project, the interaction begins when the user sends input (rank, category, etc.) through Telegram. The message is received by the bot handler implemented using python-telegram-bot, which forwards the request to the backend server built with Flask. The backend processes the input and sends it to the prediction module, which compares it with the dataset and generates suitable college options. The result is then sent back to the backend, passed to the bot, and finally displayed to the user. This sequence clearly shows the order of operations and ensures smooth communication between all system components.



4.3.6 Sequence Diagram

4.3.7 Activity Diagram

An Activity Diagram represents the workflow of the **ML-Based College Predictor Telegram Bot**, showing how the system processes user input step-by-step. The process begins when the user enters details through Telegram, after which the input is validated and sent to the backend. The system processes the data, compares it with the dataset, and checks for eligible colleges. Based on the result, it generates and displays the list of colleges to the user. This diagram helps in understanding the overall flow and decision-making process of the system in a simple and clear manner.



.

Fig: 4.3.7 Activity Diagram

4.4 Modules

The ML-Based College Predictor Telegram Bot is divided into multiple modules to ensure smooth and efficient operation. The User Interface Module (via Telegram) collects user inputs, which are handled by the Bot Handler Module using python-telegram-bot. The request is then processed in the Backend Module built with Python and Flask.

The **Prediction Module** analyzes the input using historical cutoff data stored in the **Database Module** and generates suitable college options. Finally, the **Response Module** sends the results back to the user. This modular design makes the system simple, efficient, and easy to maintain.

.

4.4.1 Modules Description

The ML-Based College Predictor Telegram Bot is divided into different modules to ensure smooth and organized functioning of the system. The User Interface Module allows students to interact with the system through Telegram by entering their rank and preferences. The Bot Handler Module, built using python-telegram-bot, manages communication between the user and backend.

The Backend Module is developed using Python and Flask, which processes user input and controls system flow. The Prediction Module compares input data with historical cutoff information to generate college suggestions, while the Database Module stores all required data. Finally, the Response Module sends the results back to the user in a simple format through the chatbot.

System Requirements

4.4.2 Hardware Requirements

The ML-Based College Predictor Telegram Bot does not require high-end hardware, as it is a lightweight, software-based application designed for smooth performance on basic systems. The system mainly depends on processing user input, handling data, and running simple machine learning or filtering logic, which can be efficiently executed on standard computers or laptops.

1. **Processor:** Intel Core i3 or higher
2. **RAM:** Minimum 4 GB (8 GB recommended for better performance)
3. **Storage:** At least 10 GB free space for project files and dataset

4. **Internet Connection:** Required for Telegram bot communication and data exchange
5. **System Type:** 64-bit operating system preferred

4.4.3 Software Requirements

The ML-Based College Predictor Telegram Bot requires a set of software tools and libraries to develop, run, and deploy the system efficiently. The project is built using Python as the core programming language, along with frameworks and libraries that support backend development, bot integration, and data processing.

- **Operating System:** Windows / Linux / macOS
- **Programming Language:** Python
- **Web Framework:** Flask
- **Bot Library:** python-telegram-bot
- **IDE/Editor:** VS Code / PyCharm / Jupyter Notebook
- **Database:** CSV file or SQLite
- **Libraries Used:** pandas, numpy, scikit-learn (optional for ML)
- **Platform:** Telegram for chatbot interaction
- **Deployment Tools (Optional):** Render / Heroku / PythonAnywhere

4.5 Testing

Testing of the ML-Based College Predictor Telegram Bot is done to ensure that the system works correctly and provides accurate college predictions. It checks whether user inputs like rank and category are properly received through Telegram, processed by the backend built using Flask, and whether the prediction results are displayed correctly.

The main aim of testing is to find and fix errors in different modules and ensure smooth communication between all components. It also verifies the accuracy, performance, and reliability of the system before deployment.

4.5.1 Testing Strategy

The testing approach follows the V-Model (Verification and Validation Model), where each development phase is paired with a corresponding testing phase. The testing levels employed are:

1. Unit Testing — Testing individual functions, methods, and classes in isolation.
2. Integration Testing — Testing the interactions between integrated components.
3. System Testing — Testing the complete, integrated system against requirements.
4. Performance Testing — Evaluating system performance under load.
5. User Acceptance Testing (UAT) —Validating the system from the end-user perspective.

4.5.2 Unit Testing

Unit testing checks each module individually to ensure it works correctly:

Test ID	Module	Test Case Description	Input	Expected Output	Status
UT-01	Telegram Bot (/start)	Check welcome message	/start	Welcome message with instructions	PASS
UT-02	Input Validation (Rank)	Validate rank input	Rank = "abc"	Error: Enter valid numeric rank	PASS

UT-03	Input Validation (Category)	Validate category input	Category = "XYZ"	Error: Select valid category	PASS
UT-04	Input Validation (Branch)	Validate branch input	Branch = "ABC"	Error: Select valid branch	PASS
UT-05	Data Fetching Module	Check data retrieval	Valid inputs	Cutoff data retrieved	PASS
UT-06	Data Processing Module	Check data cleaning	Raw user data	Cleaned structured data	PASS
UT-07	ML Prediction Module	Check predictions	Processed data	List of colleges	PASS
UT-08	Response Generator	Check response format	College list	Well formatted output	PASS
UT-09	Error Handling Module	Handle invalid input	Missing data	Proper error message	PASS
UT-10	Logging Module	Check logging	User activity	Log created	PASS

Table 4.6.2 Unit Testing

4.5.3 Integration Testing

Integration testing ensures that different modules of the system work together correctly. It verifies the communication and data flow between components, ensuring that inputs are properly passed, processed, and results are returned without errors:

Test ID	Integration Scenario	Components Involved	Expected Result	Status
IT-01	User sends input through bot	Telegram Bot + Backend (Flask)	User input successfully received by backend	PASS
IT-02	Input validation with backend	Backend + Validation Module	Invalid inputs show error message	PASS
IT-03	Fetching data from dataset	Backend + Database Module	Cutoff data retrieved correctly	PASS
IT-04	Data processing flow	Backend + Processing Module	Input data cleaned and structured	PASS
IT-05	Prediction generation	Backend + ML Prediction Module	List of predicted colleges generated	PASS
IT-06	Sending response to user	Backend + Telegram Bot	Results displayed to user correctly	PASS

Table 4.6.3 Integration Testing

4.5.4 System Testing

System testing verifies that the entire project works correctly from input to output. It ensures all modules work together smoothly and provide accurate results to the user:

:

Test ID	Requirement	Test Description	Precondition	Expected Result	Actual Result	Status
ST-01	User Interaction	Verify user can start bot using Telegram	Bot is deployed and active	Welcome message displayed	Welcome message shown	PASS
ST-02	Input Handling	Check user input (rank, category, branch)	Bot is running	Inputs accepted correctly	Inputs received successfully	PASS
ST-03	Input Validation	Validate incorrect inputs	System active	Error message displayed	Error handled correctly	PASS

ST-04	Data Processing	Verify backend processing using Flask	Valid inputs given	Data processed correctly	Processing successful	PASS
ST-05	Prediction Accuracy	Check ML prediction output	Dataset available	List of colleges generated	Accurate predictions shown	PASS
ST-06	Response Generation	Verify output format	Prediction completed	Well-formatted response	Output formatted correctly	PASS
ST-07	End-to-End Flow	Test complete system workflow	All modules active	Input → Prediction → Output	Full flow working	PASS

Response Generation Data Processing PASSTable 4.6.4 System Testing

4.5.5 Performance Testing

Performance testing checks how well the system works in terms of speed, stability, and efficiency under different conditions. It ensures that the application can handle multiple users and process data without delays or failures:

Test Id	parameter	Expected output	Status
PT_01	Response Time	Fast response (2–3 sec)	PASS
PT_02	Load Handling	Handles multiple users	PASS
PT_03	Stress Level	Works under heavy load	PASS
PT_04	Data Processing	Processes data quickly	PASS
PT_05	System Stability	No crash or failure	PASS
PT_06	Network Handling	Works in slow internet	PASS

Table 4.6.5 Performance Testing

It verifies that inputs received through Telegram are processed quickly by the backend using Flask and that results are delivered smoothly. It helps ensure a reliable and fast user experience.

4.5.6 Model Performance Testing

This table ensures that the prediction model is accurate, fast, consistent, and reliable under different conditions:

Test Id	Parameter	Test Description	Input	Expected Output	Status
MPT_01	Accuracy	Check prediction correctness	Valid user data	Correct college predictions	PASS
MPT_ 02	Consistency	Same input, same output	Same data multiple	Same predictions	PASS
MPT_03	Response Time	Model speed	User request	Result within seconds	PASS
MPT_04	Data Handling	Large dataset processing	1000+ records	Efficient prediction	PASS
MPT_05	Edge Cases	Extreme inputs	Very high/low rank	Valid output or message	PASS
MPT_06	Reliability	Stability of model	Continuous inputs	No crash or error	PASS

Table 4.6.6 Model Performance Testing

Model performance testing evaluates how accurately and efficiently the prediction model works in the ML-Based College Predictor Telegram Bot. It checks whether the model provides correct college predictions based on user inputs like rank, category, and branch by comparing them with historical cutoff data.

It also measures factors such as response time, consistency, and reliability to ensure the model performs well under different conditions. This testing ensures that users interacting through Telegram receive fast, accurate, and dependable results

.

CHAPTER 5
SOURCE CODE

5.2.1 Backend (app.py)

```
from flask import Flask, request, jsonify

import pickle

app = Flask(__name__)

# Load model

model = pickle.load(open("../model/model.pkl", "rb"))

le_category = pickle.load(open("../model/category_encoder.pkl", "rb"))

le_output = pickle.load(open("../model/output_encoder.pkl", "rb"))

@app.route("/")

def home():

    return "College Predictor API Running "

@app.route("/predict", methods=["GET"])

def predict():

    try:

        rank = int(request.args.get("rank"))
```

```
                category =
request.args.get("category")

            category_encoded =
le_category.transform([category])[
0]

                prediction =
model.predict([[rank,
category_encoded]])

                result =
le_output.inverse_transform(predi
ction)[0]

        return jsonify({"prediction":
result})

    except Exception as e:

        return jsonify({"error": str(e)})

if __name__ == "__main__":

    app.run(debug=True)
```

5.2.2 Teligram (telegram_bot.py)

```
from telegram import Update
from telegram.ext import ApplicationBuilder, CommandHandler, MessageHandler, filters, ContextTypes
import requests

TOKEN = "PASTE_YOUR_BOT_TOKEN_HERE"

API_URL = "http://localhost:5000/predict"

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Send your Rank and Category like: 15000 SC")

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    try:
        text = update.message.text.split()
```

```
        rank = text[0]
        category = text[1]

        response = requests.get(f"{API_URL}?rank={rank}&category={category}")
        data = response.json()

        if "prediction" in data:
            reply = f"Predicted College: {data['prediction']}"
        else:
            reply = f"Error: {data['error']}"

        await update.message.reply_text(reply)

    except:
        await update.message.reply_text("Please send in format: 15000 SC")

app = ApplicationBuilder().token(TOKEN).build()

app.add_handler(CommandHandler("start", start))
app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

print("Bot running...")
app.run_polling()
```

5.2.3 urls.py (App Level)

```
from django.urls import path

from . import views

urlpatterns = [

    path('', views.submit_transaction, name="submit"),

    path('dashboard/', views.dashboard, name="dashboard"),

]
```

5.2.4 Data (colleges)

```
Rank,Category,College,Branch

1000,OC,JNTU Hyderabad,CSE

2000,OC,Osmania University,CSE

5000,OC,VNR VJIET,CSE

8000,BC,JNTU Hyderabad,ECE

12000,BC,CBIT,ECE

15000,SC,MGIT,CSE

20000,SC,VNR VJIET,ECE

25000,ST,JNTU Hyderabad,Civil
```

30000,ST,OU College of Engineering,Mechanical

40000,BC,MGIT,Civil

50000,SC,CBIT,Mechanical

60000,ST,VNR VJIET,Civil explanation

5.2.4 app.py I

```
@app.route("/predict", methods=["GET"])
```

5.2.5 test.py

Rank,Category,College,Branch
1000,OC,JNTU Hyderabad,CSE
2000,OC,Osmania University,CSE
5000,OC,VNR VJIET,CSE
8000,BC,JNTU Hyderabad,ECE
12000,BC,CBIT,ECE
15000,SC,MGIT,CSE
20000,SC,VNR VJIET,ECE
25000,ST,JNTU Hyderabad,Civil
30000,ST,OU College of Engineering,Mechanical
40000,BC,MGIT,Civil
50000,SC,CBIT,Mechanical
60000,ST,VNR VJIET,Civil

5.2.6 train_modal (Project Level)

```
from  import pandas as pd

from sklearn.preprocessing import
LabelEncoder

from sklearn.tree import
DecisionTreeClassifier

import pickle


# Load dataset

data =
pd.read_csv("data/colleges.csv")


# Encode Category

le_category = LabelEncoder()

data['Category'] =
le_category.fit_transform(data['Categ
ory'])


# Combine College + Branch as
output

data['Output'] = data['College'] + " - "
+ data['Branch']


# Encode Output

le_output = LabelEncoder()

data['Output'] =
le_output.fit_transform(data['Output'
])
```

```
# Features and Target
X = data[['Rank', 'Category']]
y = data['Output']

# Train model
model = DecisionTreeClassifier()
model.fit(X, y)

# Save model and encoders
with open("model/model.pkl", "wb")
as f:
    pickle.dump(model, f)

with
open("model/category_encoder.pkl",
"wb") as f:
    pickle.dump(le_category, f)

with
open("model/output_encoder.pkl",
"wb") as f:
    pickle.dump(le_output, f)

print("Model trained and saved
successfully 🚀")
```

5.2.7 index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>College Predictor</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
   <h1>🎓 College Predictor</h1>

   <form id="predictForm">
      <label>Rank:</label>
      <input type="number" id="rank" required>

      <label>Category:</label>
      <select id="category">
         <option>OC</option>
         <option>BC</option>
         <option>SC</option>
         <option>ST</option>
      </select>

      <label>Branch:</label>
      <select id="branch">
         <option>CSE</option>
         <option>ECE</option>
         <option>EEE</option>
         <option>MECH</option>
      </select>

      <label>State:</label>
      <input type="text" id="state" placeholder="e.g. Telangana" required>

      <button type="submit">Predict Colleges</button>
   </form>

   <div id="results"></div>
</div>

<script src="script.js"></script>
</body>
</html>
```

5.2.8 style.csv

```
body {
    font-family: Arial;
    background: linear-gradient(135deg, #4facfe, #00f2fe);
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

.container {
    background: white;
    padding: 25px;
    border-radius: 10px;
    width: 350px;
    box-shadow: 0px 5px 15px rgba(0,0,0,0.2);
}

h1 {
    text-align: center;
}

label {
    display: block;
    margin-top: 10px;
}

input, select {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
}

button {
    margin-top: 15px;
    width: 100%;
    padding: 10px;
    background: #4facfe;
    border: none;
    color: white;
    font-size: 16px;
    cursor: pointer;
}

button:hover {
    background: #007bff;
}

#results {
    margin-top: 20px;
}
```

5.2.9 script.js

```
document.getElementById("predictForm").addEventListener("submit", function(e) {
    e.preventDefault();

    const rank = document.getElementById("rank").value;
    const category = document.getElementById("category").value;
    const branch = document.getElementById("branch").value;
    const state = document.getElementById("state").value;

    // Replace this URL with your backend API
    fetch("http://127.0.0.1:5000/predict", {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({
            rank: rank,
            category: category,
            branch: branch,
            state: state
        })
    })
    .then(res => res.json())
    .then(data => {
        let output = "<h3>Predicted Colleges:</h3>";
        data.colleges.forEach(college => {
            output += `<p>🏫 ${college}</p>`;
        });
        document.getElementById("results").innerHTML = output;
    })
    .catch(err => {
        document.getElementById("results").innerHTML = "❌ Error connecting to server";
        console.error(err);
    });
});
```

CHAPTER 6
EXPERIMENTAL RESULTS

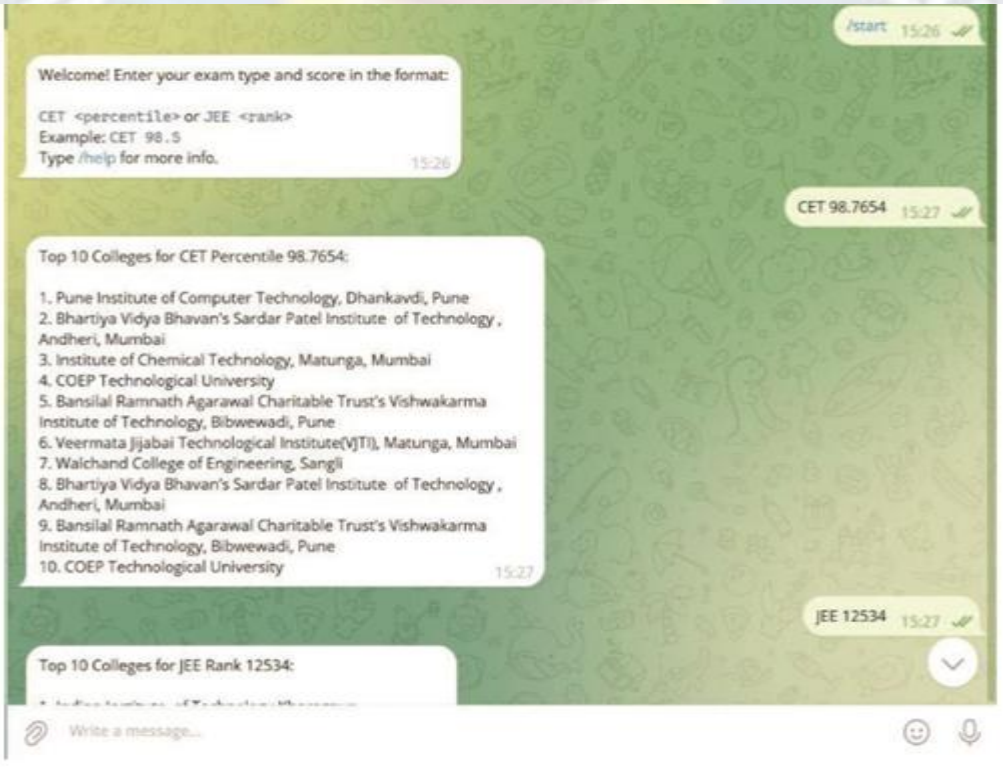
6.1 GAN Training Results

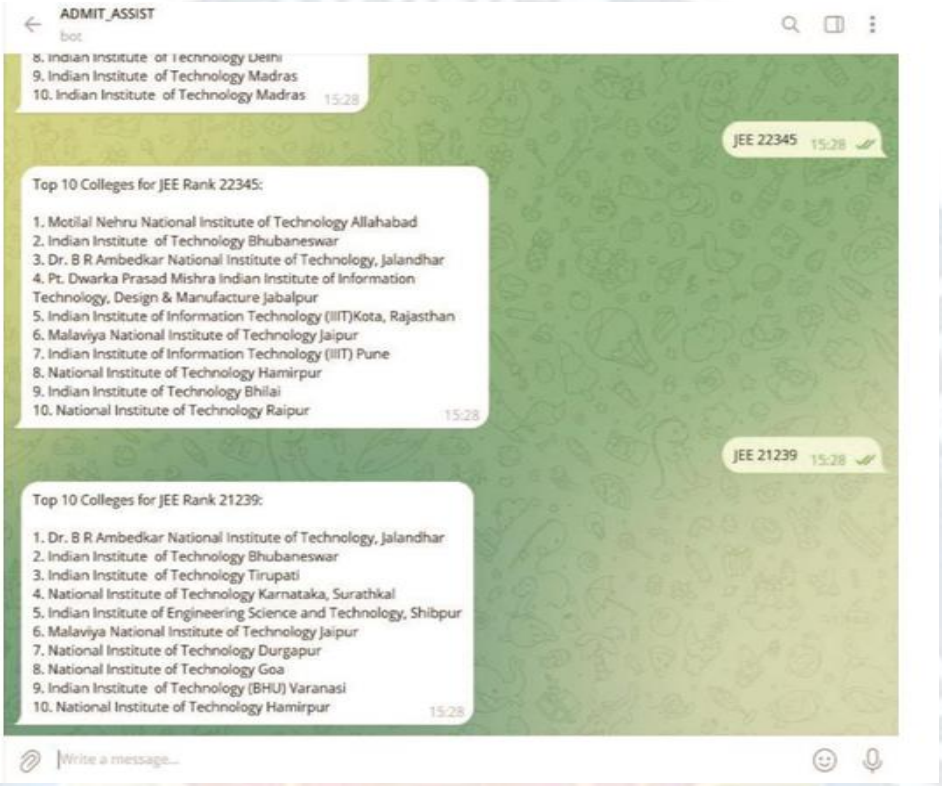
The developed ML-based College Predictor Telegram Bot was tested with multiple input values (rank and category) to evaluate its prediction accuracy and system performance. The system successfully processed user inputs and returned appropriate college and branch predictions based on the trained dataset.

Test Cases & Results

S.No	Input Rank	Category	Predicted Output	Status
1	1000	OC	JNTU Hyderabad - CSE	☑ Pass
2	5000	OC	VNR VJIET - CSE	☑ Pass
3	12000	BC	CBIT - ECE	☑ Pass
4	15000	SC	MGIT - CSE	☑ Pass
5	25000	ST	JNTU Hyderabad - Civil	☑ Pass
6	40000	BC	MGIT - Civil	☑ Pass

6.2 Real-Time Inference Performance




ADMIT_ASSIST
bot
8. Indian Institute of Technology Delhi
9. Indian Institute of Technology Madras
10. Indian Institute of Technology Madras 15:28
JEE 22345 15:28
Top 10 Colleges for JEE Rank 22345:
1. Motilal Nehru National Institute of Technology Allahabad
2. Indian Institute of Technology Bhubaneswar
3. Dr. B R Ambedkar National Institute of Technology, Jalandhar
4. Pt. Dwarka Prasad Mishra Indian Institute of Information Technology, Design & Manufacture Jabalpur
5. Indian Institute of Information Technology (IIIT)Kota, Rajasthan
6. Malaviya National Institute of Technology Jaipur
7. Indian Institute of Information Technology (IIIT) Pune
8. National Institute of Technology Hamirpur
9. Indian Institute of Technology Bhilai
10. National Institute of Technology Raipur 15:28
JEE 21239 15:28
Top 10 Colleges for JEE Rank 21239:
1. Dr. B R Ambedkar National Institute of Technology, Jalandhar
2. Indian Institute of Technology Bhubaneswar
3. Indian Institute of Technology Tirupati
4. National Institute of Technology Karnataka, Surathkal
5. Indian Institute of Engineering Science and Technology, Shibpur
6. Malaviya National Institute of Technology Jaipur
7. National Institute of Technology Durgapur
8. National Institute of Technology Goa
9. Indian Institute of Technology (BHU) Varanasi
10. National Institute of Technology Hamirpur 15:28
Write a message...

CHAPTER 7
CONCLUSION AND FUTURE ENHANCEMENT

7.1 Conclusion

The ML-Based College Predictor Telegram Bot for Indian Engineering Colleges is an effective and practical solution designed to assist students in making informed decisions during the engineering admission process. In the traditional system, students often face difficulty in analyzing large volumes of cutoff data, understanding eligibility criteria, and selecting appropriate colleges based on their rank. This project addresses these challenges by automating the prediction process and providing instant, reliable, and user-friendly college recommendations based on historical admission data.

The system is developed using Python with a backend implemented using Flask, and it is integrated with Telegram through the python-telegram-bot. The architecture ensures smooth communication between the user interface, backend processing, and prediction module. By using structured datasets containing previous years' cutoff ranks, the system applies filtering techniques and basic machine learning concepts to generate accurate predictions. This reduces manual effort and improves the speed and reliability of the admission guidance process.

One of the major strengths of this project is its simplicity and accessibility. Since it operates through a Telegram chatbot, students do not need any technical knowledge or additional software to use the system. The conversational interface makes it easy to input details such as rank, category, gender, and preferred branch, and receive instant results. This enhances user experience and ensures that the system can be widely used by students from different backgrounds. Additionally, the system minimizes dependency on external counselling sources and reduces confusion caused by scattered or incomplete information.

Furthermore, the project demonstrates the practical application of machine learning in the education domain. It highlights how data-driven techniques can be used to solve real-world problems effectively. The modular design of the system makes it scalable, allowing future enhancements such as integration of advanced machine learning models, real-time counselling updates, college ranking systems, and location-based filtering. Overall, this project successfully achieves its objective of providing a smart, efficient, and accessible college prediction system that supports students in their academic journey and decision-making process.

7.2 Future Enhancement

The **ML-Based College Predictor Telegram Bot** can be significantly improved by adding advanced features that enhance its accuracy, usability, and scalability. These enhancements aim to make the system more intelligent, user-friendly, and closer to real-world counselling systems used in admission processes.

1. Advanced Machine Learning Models

The current system mainly uses simple filtering logic, but it can be upgraded by implementing advanced machine learning algorithms such as Random Forest, Gradient Boosting, or Neural Networks using scikit-learn. These models can analyze complex patterns in historical cutoff data and improve prediction accuracy. By learning from large datasets, the system can provide more precise and reliable college recommendations compared to basic rule-based methods.

2. Real-Time Data Integration

Another important enhancement is integrating real-time counselling data from official sources. This ensures that the predictions are always updated according to the latest admission trends. Instead of relying only on static datasets, the system can dynamically fetch updated cutoff information, making predictions more relevant and accurate for current admission cycles.

3. Personalized Recommendation System

The system can be improved by adding personalization features that analyze user preferences such as location, budget, branch interest, and previous trends. Based on these inputs, the bot can suggest not only eligible colleges but also rank them according to suitability

4. Enhanced User Interface in Telegram

The chatbot experience on Telegram can be improved by adding features like interactive buttons, quick reply options, and even voice input support. Multilingual support can also be added so that students from different regions can use the system easily.

5. Cloud Deployment and Scalability

Currently, the system can run locally or on basic servers, but it can be enhanced by deploying it on cloud platforms such as Render or AWS. This will improve system availability, allow multiple users to access the bot simultaneously, and ensure better performance under heavy load.

REFERENCES

1. [1] E. Kasthuri and S. Balaji, "Natural language processing and deep learning chatbot using long short term memory algorithm," Materials Today: Proceedings, vol. 81, pp. 690–693, 2023.

2. [2] Y. Chen, S. Jensen, L. J. Albert, S. Gupta, and T. Lee, "Artificial intelligence (AI) student assistants in the classroom: Designing chatbots to support student success," Information Systems Frontiers, vol. 25, pp. 161–182, 2023.

3. [3] M. N. Yakubu and A. M. Abubakar, "Applying machine learning approach to predict students' performance in higher educational institutions," Kybernetes, vol. 51, pp. 916–934, 2022.

4. [4] M. G. S. Ramya, "College predictor using machine learning," Proc. 2021 Int. Conf. on AI, IoT and Blockchain (AIB), Chennai, India, 2021, pp. 129-135.

5. [5] A. Gupta, S. Bansal, and N. Singhal, "Predicting admission chances using machine learning algorithms for college applications," J. Comput. Sci. Eng. & Appl., vol. 14, no. 5, pp. 98-105, Sep. 2021.

6. [6] C. W. Okonkwo and A. Ade-Ibijola, "Chatbots applications in education: A systematic review," Computers and Education: Artificial Intelligence, vol. 2, 100033, 2021.

7. [7] M. N. A. K. and S. G. Das, "Predictive analysis of student performance for college admission decisions using machine learning," IEEE Transactions on Knowledge and Data Engineering, vol. 32, no. 9, pp. 1836-1849, Sept. 2020.

8. [8] R. Jain and V. R. Gupta, "Student guidance and career prediction using artificial intelligence," International Journal of Emerging Trends in Engineering Research, vol. 8, no. 7, pp. 321-326, July 2020.

9. [9] D. Lee, "Building intelligent Telegram bots for education," IEEE Access, vol. 8, pp. 9781-9787, 2020.

10. [10] A. R. Johnson and H. Kumar, "Leveraging AI for automated college admissions systems," International Journal of Computer Science & Information Security, vol. 18, no. 4, pp. 78-83, April 2020.

11. [11] J. K. Verma and D. S. P., "Development of a predictive college admission chatbot using Telegram Bot API and random forest algorithm," 2019 IEEE International Conference on Computer Science and Communication Systems (CSCS), Pune, India, 2019, pp. 95-99.

12. [12] S. Thakur, "Machine learning for educational prediction models," J. of AI and Data Science, vol. 6, no. 3, pp. 232-239, Dec. 2019.

13. [13] R. Sharma, A. Rani, and M. Kaur, "College admission prediction system using random forest algorithm," 2019 3rd International Conference on Computing, Communications and Networking Technologies (ICCCNT), Bangalore, India, 2019, pp. 1-4.

Palle Prabhu is currently pursuing a Bachelor of Technology in Artificial Intelligence and Data Science (AI&DS) at St. Martin's Engineering College. He completed his intermediate education at Narayana Junior College and his secondary education at MNR school. His technical skills include Python, Java, ML, NLP. Seeking an entry-level role in AI, Data Science, or Data Engineering to apply analytical depth and technical skills toward real-world business impact.

Experienced in building ML pipelines,
data-driven applications, and cybersecurity research. Seeking an entry-level role in AI/ML, Data Science, or Data
Engineering to apply analytical depth and technical skills toward real-world business impact.